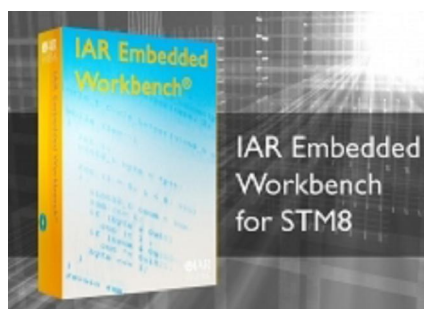


第五章 熟悉 IAR 开发环境

IAR Systems 推出开发工具“STM8 系列嵌入式设计工作台”(EWSTM8)，支持 8 位微控制器市场主流的 STM8 (STM8A, STM8L, STM8S) 系列产品。IAR EWSTM8 嵌入式设计工作台提供一整套开发工具，包括一个项目管理器、编辑器和项目创建工具 (C 语言编译器和链接器)。该工作台还为开发人员提供调试功能，可以连接意法半导体价格低廉的在线调试器 ST-LINK 以及先进的高端仿真器 STice。下面向读者详细介绍 IAR 的使用方法，希望读者能够迅速熟悉和掌握 IAR。

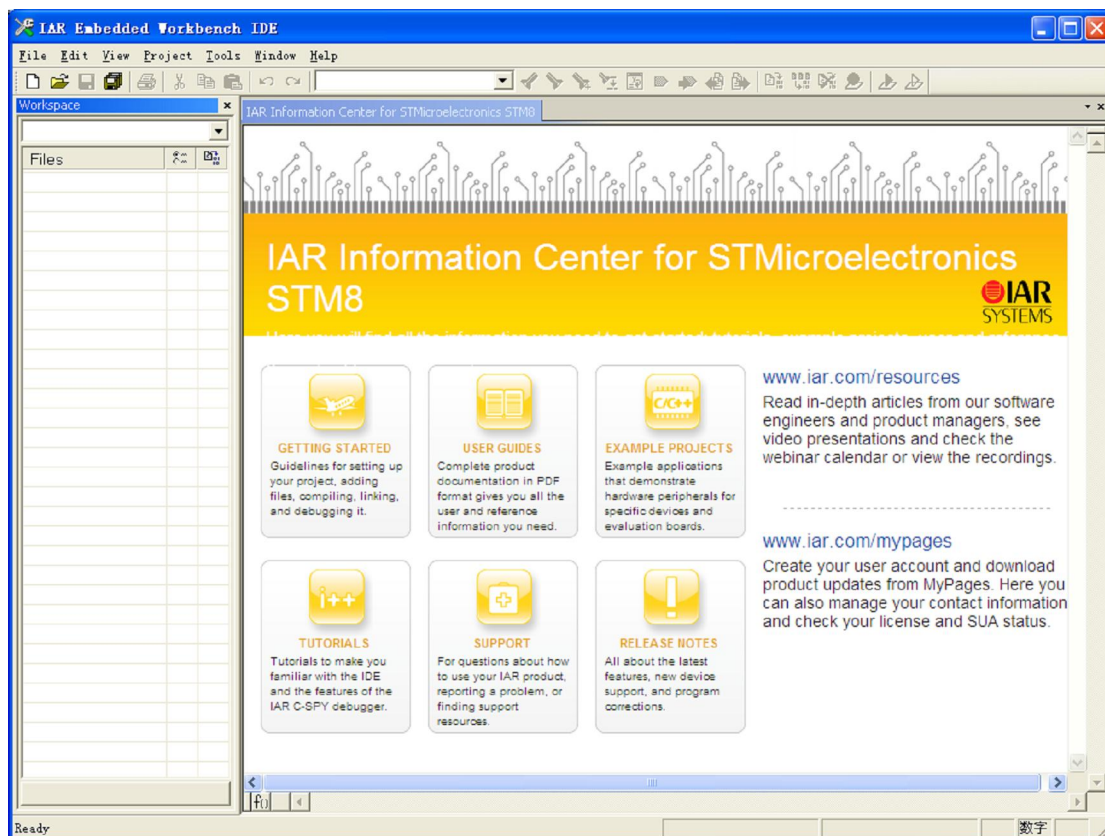
5.1 IAR For STM8 的下载以及安装



IAR For STM8 可以在 IAR 的官网中下载，现在提供两种版本，一是免费的 8K 受限制版本，另一个是评估版的 30 天免费的版本。可以在以下网址下载：

<http://www.iar.com/en/Products/IAR-Embedded-Workbench/STMicroelectronics-STM8/>

安装完成之后，运行 IAR 集成开发环境，其界面如图所示：



5.2 使用 IAR 创建 C 语言项目

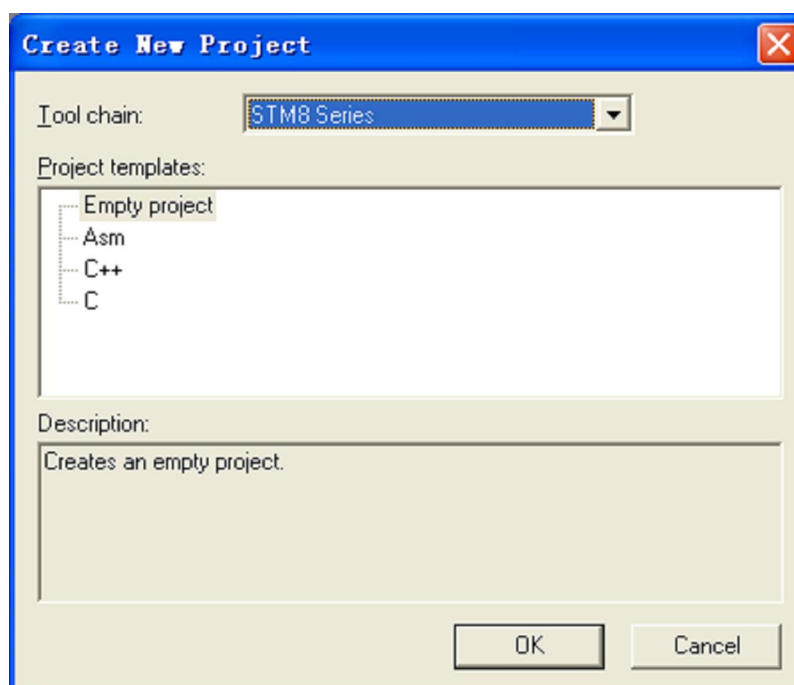
5.2.1 新建

a) 创建一个 Workspace

首先，创建一个 Workspace。选择菜单 File>New>Workspace

b) 创建一个 Project

1、创建一个新工程，选择 Project>Create New Project，创建新工程的对话框，如下图所示：

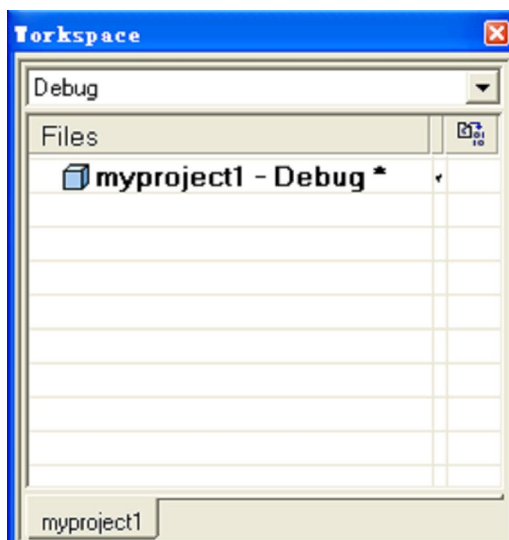


2、Tool chain 默认是 STM8 Series。无需再选择

3、在 Project templates 中选择 C 或者其它

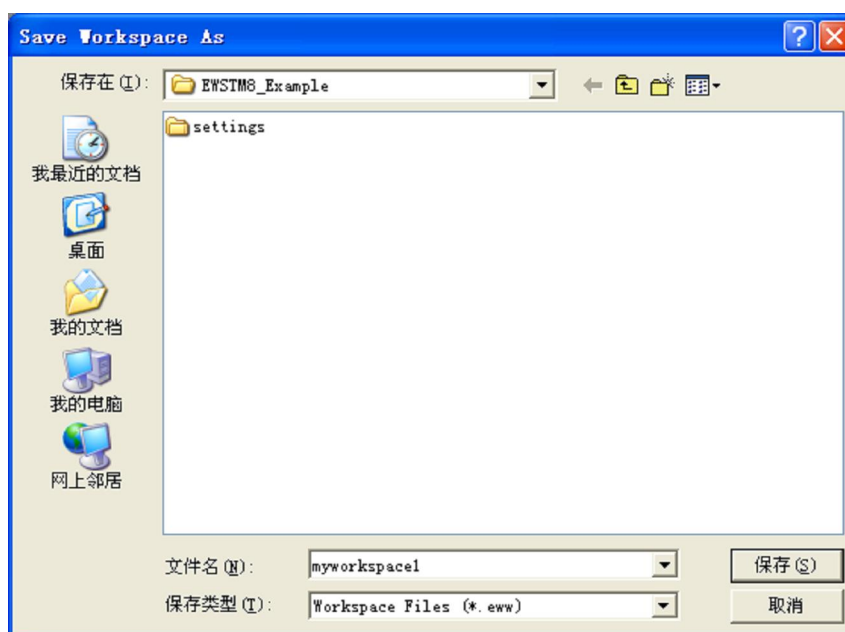
4、在弹出 Save As 对话框，选择 Project 保存的路径，并输入 Project 的名字。

在 Workspace 窗口中，显示如下：



有 Debug 和 Release 两个配置，默认配置是 Debug。

5、在添加文件到工程之前，先保存 Workspace。选择 File>Save>Workspace，指定要保持的路径，并输入 Workspace 的名字。



一个 workspace 文件的扩展名是 eww. 此文件列出了添加到 workspace 中的所有的项目。相关当前会话信息，比如 windows 的保存路径和断点保存于 projects\settings 目录下。

c) 添加文件到工程

可选择 Project>Add Files 选择要添加的 C 文件。找到相应 MCU 型号的头文件，拷贝

到当前的工程目录中。默认地，头文件在 IAR 安装目录 inc 下：C:\Program Files\IAR Systems\Embedded Workbench 6.0 Kickstart\stm8\inc。

当然如果是 IAR 集成环境默认的头文件，可以直接添加头文件就可以。不用把头文件复制到工程目录中。

本例中使用了 iostm8s207rb.h。编写一个简单的 C 程序：

```
#include "iostm8s207rb.h"

#define LED1_FLASH PD_ODR_ODR0 = !PD_ODR_ODR0

void CLK_Init(void);
void GPIO_Init(void);
void TIM1_Init(void);

void main( void )
{
    CLK_Init();
    GPIO_Init();
    TIM1_Init();
    asm("rim");
    // 主循环里没有程序需要执行
    while(1);
}

void CLK_Init(void)
{
    CLK_CKDIVR = 0x00; // 16M内部RC直接输出
}

void GPIO_Init(void)
{
    PD_DDR = 0x01; // 配置PD端口的方向寄存器PD0输出
    PD_CR1 = 0x01; // 设置PD0为推挽输出
    PD_ODR = 0xFF;
}

void TIM1_Init(void)
{
    TIM1_PSCRH = 0x3E; // 16M系统时钟经预分频f=fck/(PSCR+1)
    TIM1_PSCRL = 0x7F; // PSCR=0x3E7F, f=16M/(0x3E7F+1)=1000Hz, 每个计数周期1ms
    TIM1_ARRH = 0x01; // 自动重载寄存器ARR=0x01F4=500
    TIM1_ARRL = 0xF4; // 每计数500次产生一次中断, 即500ms
    TIM1_IER = 0x01; // 允许更新中断
    TIM1_CR1 = 0x01; // 计数器使能, 开始计数
}

#pragma vector=TIM1_OVR_UIF_vector //0x0D
__interrupt void TIM1_OVR_UIF(void)
{
    LED1_FLASH;
    TIM1_SR1 = 0; // 清除更新中断标记, 这步不能漏掉, 否则会连续进入中断程序
}
```

中断

声明一个中断向量, 说明如下:

```
#pragma vector=TIM1_OVR_UIF_vector //0x0D
__interrupt void TIM1_OVR_UIF(void)
{
    LED1_FLASH;
    TIM1_SR1 = 0; // 清除更新中断标记, 这步不能漏掉, 否则会连续进入中断程序
}
```

说明:

#pragma vector: 是 IAR 中断向量指令

=0x0D: 其数字代表中断向量编号。STM8 的地址是从 0x008000 开始, IAR 的中断编号从 0 开始。中断向量号一次按照中断地址递增。如:

复位向量是 0x008000，中断向量号是 0x00

TRAP 的中断地址是 0x008004，中断向量号是 0x01

`__interrupt void TIM1_OVR_UIF(void)`：是声明一个中断函数，主要 interrupt 是两个下划线，TIM1_OVR_UIF 是中断函数名字，可以自己定义。

Interrupt vector mapping

STM8S105xx

7 Interrupt vector mapping

Table 11. Interrupt mapping

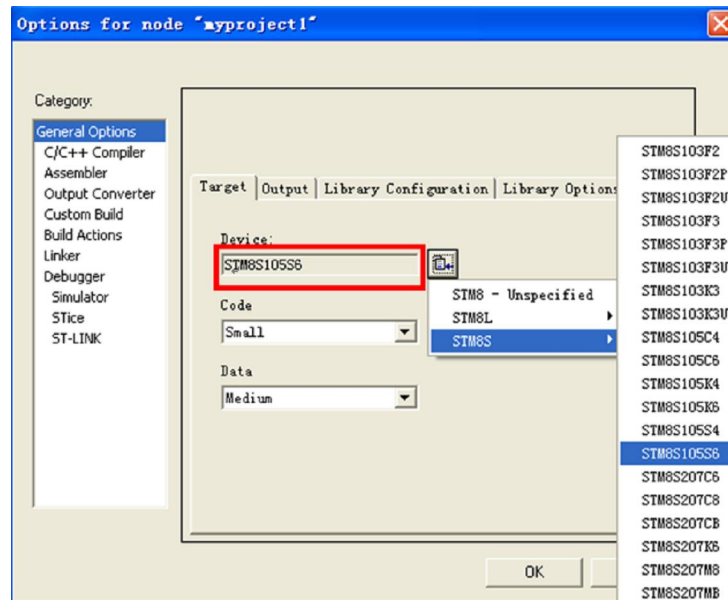
IRQ no.	Source block	Description	Wakeup from halt mode	Wakeup from active-halt mode	Vector address
	RESET	Reset	Yes	Yes	0x00 8000
	TRAP	Software interrupt	-	-	0x00 8004
0	TLI	External top level interrupt	-	-	0x00 8008
1	AWU	Auto wake up from halt	-	Yes	0x00 800C
2	CLK	Clock controller	-	-	0x00 8010
3	EXTI0	Port A external interrupts	Yes ⁽¹⁾	Yes ⁽¹⁾	0x00 8014
4	EXTI1	Port B external interrupts	Yes	Yes	0x00 8018
5	EXTI2	Port C external interrupts	Yes	Yes	0x00 801C
6	EXTI3	Port D external interrupts	Yes	Yes	0x00 8020
7	EXTI4	<code>#pragma vector=0x09</code>			0x00 8024
8		<code>interrupt void EXTI4_IRQHandler(void)</code>			0x00 8028
9					0x00 802C
10	SPI	End of transfer	Yes	Yes	0x00 8030
11	TIM1	TIM1 update/overflow/underflow/trigger/break	-	-	0x00 8034
12	TIM1	TIM1 capture/compare	-	-	0x00 8038
13	TIM2	TIM2 update /overflow	-	-	0x00 803C
14	TIM2	TIM2 capture/compare	-	-	0x00 8040
15	TIM3	Update/overflow	-	-	0x00 8044
16	TIM3	Capture/compare	-	-	0x00 8048
17		Reserved	-	-	0x00 804C
18		Reserved	-	-	0x00 8050
19	I ² C	I ² C interrupt	Yes	Yes	0x00 8054
20	UART2	Tx complete	-	-	0x00 8058
21	UART2	Receive register DATA FULL	-	-	0x00 805C
22	ADC1	ADC1 end of conversion/analog watchdog interrupt	-	-	0x00 8060
23	TIM4	TIM4 update/overflow	-	-	0x00 8064
24	Flash	EOP/WR_PG_DIS	-	-	0x00 8068
Reserved					0x00 806C to 0x00 807C

由此可以知道 TIM1 的溢出是 11 号，加上 2 就是 13，转为 16 进制就是 0x0D

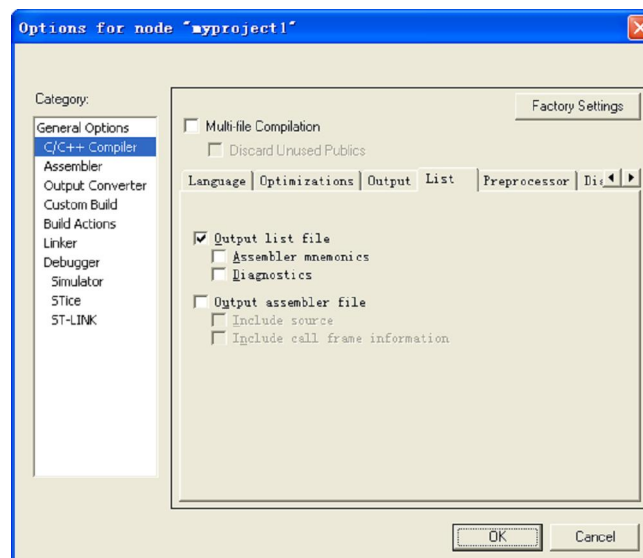
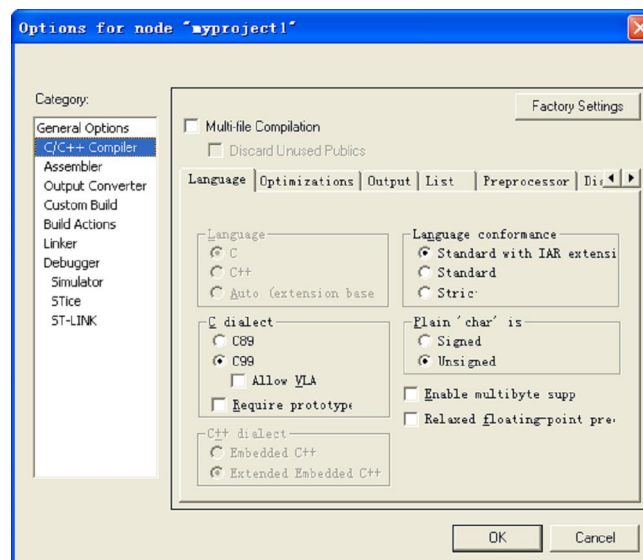
d) 工程选项配置

选择 Project>Options，或者在 Workspace 窗口，选中 project 名字，右击选择选择 “Options...”

1) 在 Category 中，选择 “General Options”，如图：在 Target 的 Device 中，选择相应的 MCU 型号。其他选择默认。

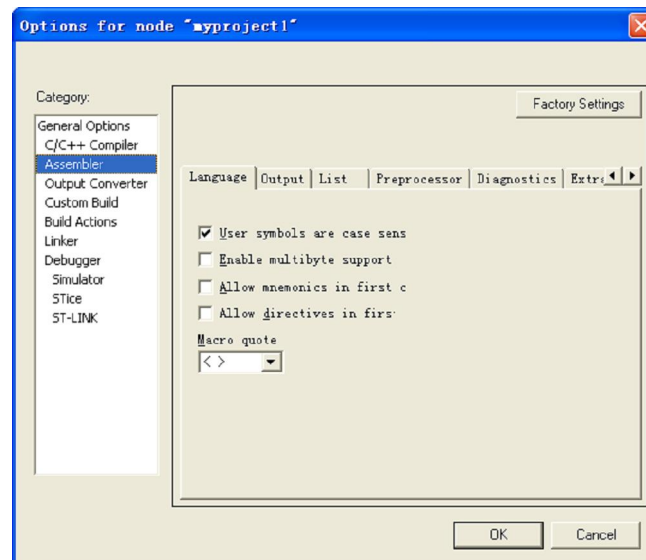


2) 在 Category 中, 选择 C/C++ Compiler, 显示 compiler 选项页

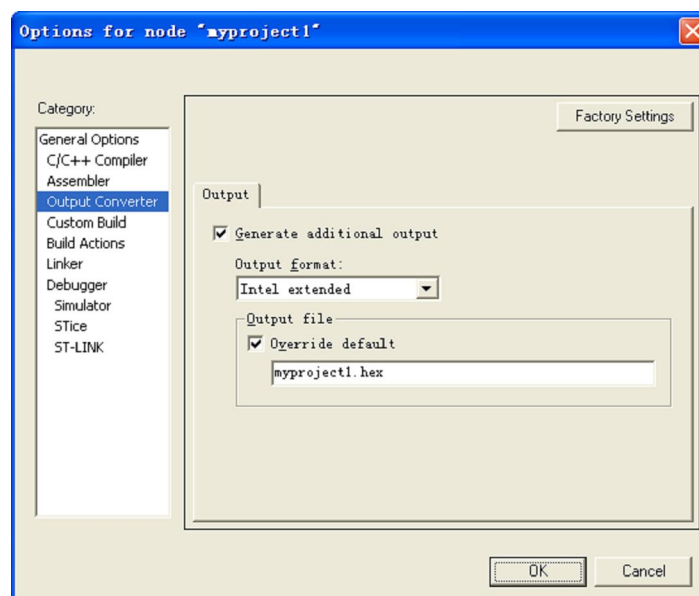


选择 Output list file, 输出列表文件

3) 在 category 中, 选择 Assembler, 显示 Assmbler 选项页

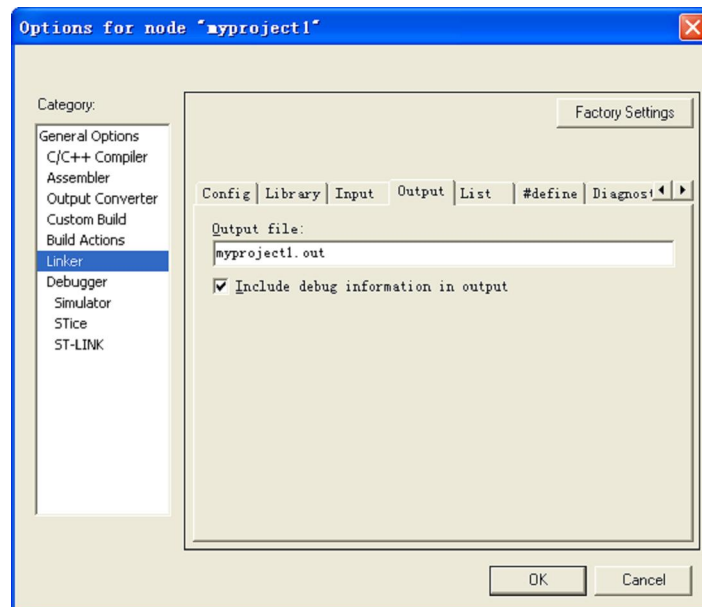


4) 在 category 中, 选择 Output Converter, 显示 Output Converter 选项页

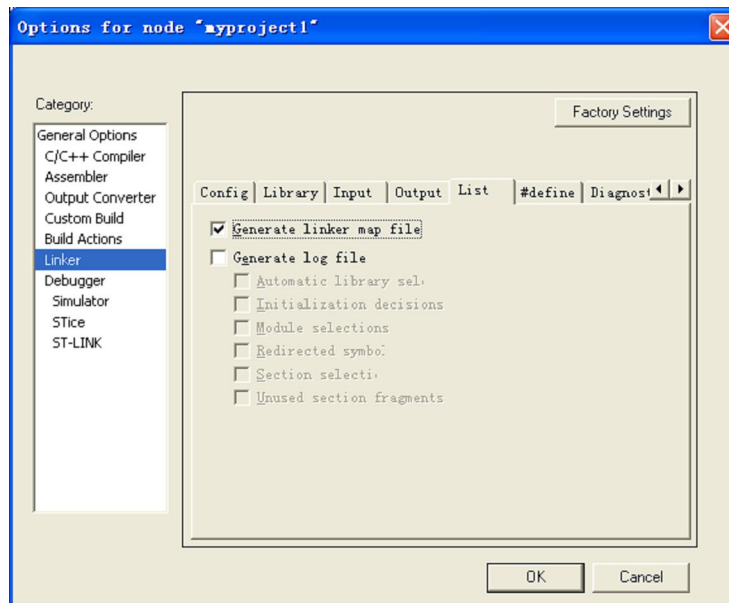


在 Debug\Exe 目录中, 产生目标文件。IAR 默认的是 .out 文件。此界面可选择不同的格式目标文件。本例选择 Intel extended 格式的目标文件。

5) 在 Category 中, 选择 Linker, 显示 Linker 选项页

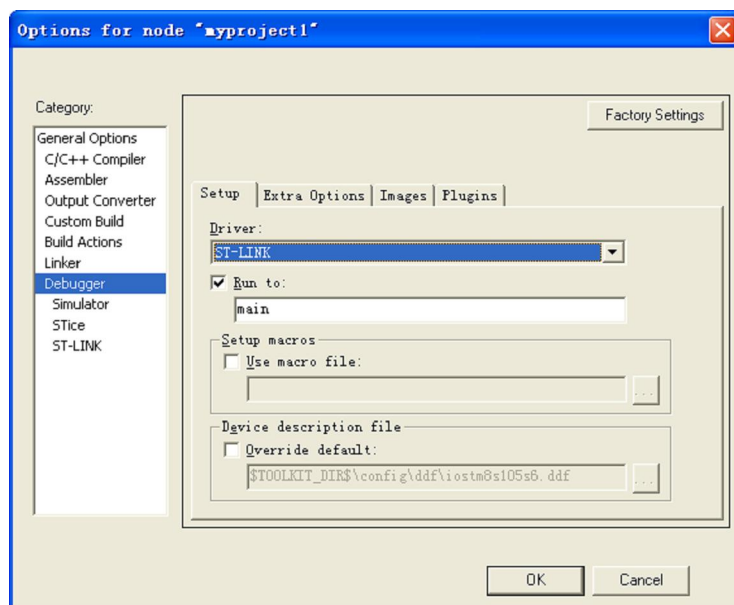


默认输出文件是：工程名字.out



选中 Generate linker map file, 输出工程的 map 文件。

6) 在 Category 中，选择 Debugger，显示 Debugger 选项页

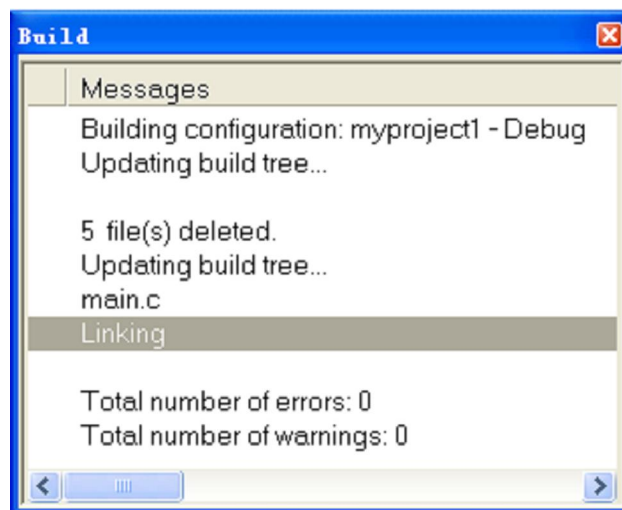


在 Driver 中，选择 ST-LINK。IAR 目前还支持软件仿真模拟和 STice 工具

5.2.2 编译

通过 Project>Compile 或者 Project>Rebuild All 来编译某个源文件或者全部重新编译。

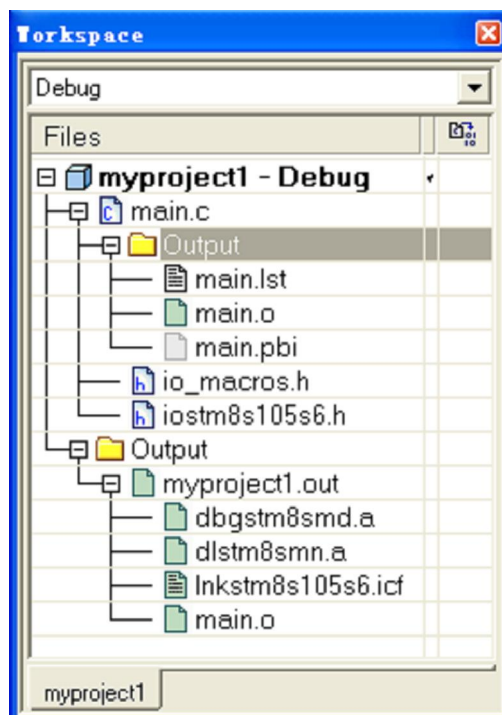
编译创建信息显示窗口如下：



IDE 将创建 List, Obj 和 Exe 目录。

- List 目录是列表文件的目录。其扩展名为.lst
- Objm 目录是目标文件目录。ILINK 连接器的扩展名为.o
- Exe 目录是可执行文件目录。

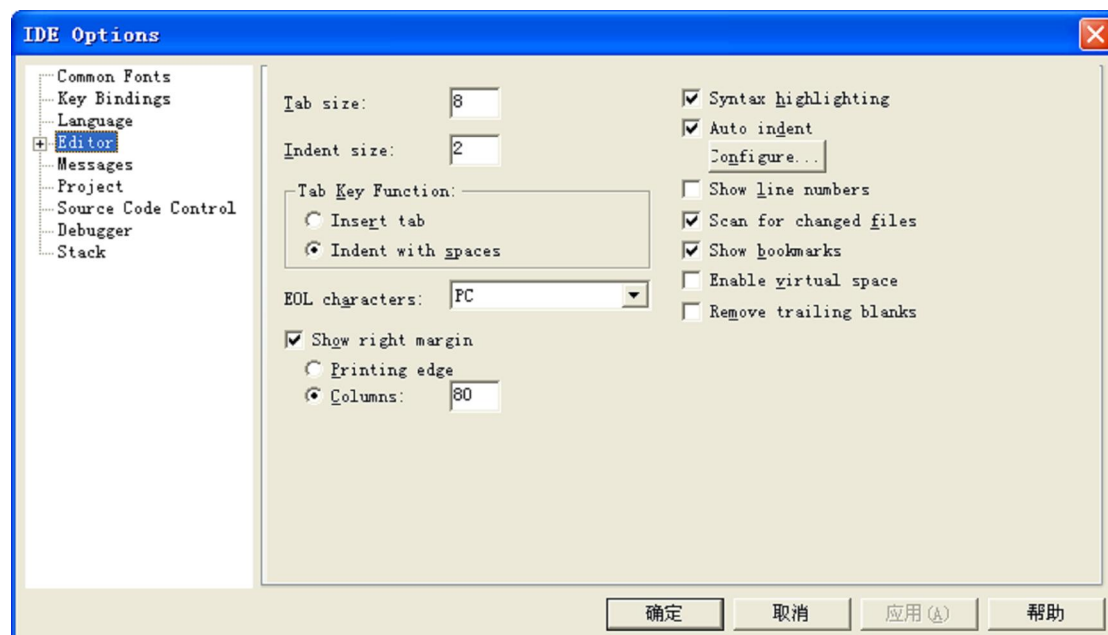
可通过 workspace 中的 output 文件夹，来查看所有输出的文件。



查看列表文件

List 文件可查看汇编代码和每个段的二进制代码。还显示变量如何分配。List 文件的结

尾现实了堆栈，代码和数据存储器的大小。可选择 Tools>Options 来配置更新改变的文件



若要看代码的大小，可找到工程目录下的 \Debug\List，在 list 下有.map 文件。用记事

本打开.map 文件，在.map 的末尾，可查看程序代码大小。

如本例中的.map 文件。

```
...
[1] = D:\Tutorial\EWSTM8S\Debug\Obj
[2] = command line
[3] = dbgstm8smd.a
[4] = dlstm8smn.a

294 bytes of readonly code memory
132 bytes of readonly data memory
274 bytes of readwrite data memory

Errors: none
Warnings: none
...
```

本例的程序代码是：294 个字节

5.2.3 调试

选择 Project>Download and Debug. 进入调试状态。或者选择工具栏 

使用 Auto 窗口

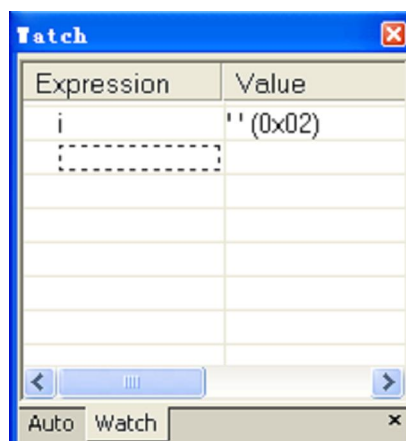
在调试状态下，选择 View>Auto 打开 Auto 窗口。Auto 窗口自动地显示当前的修改变量。如

图：



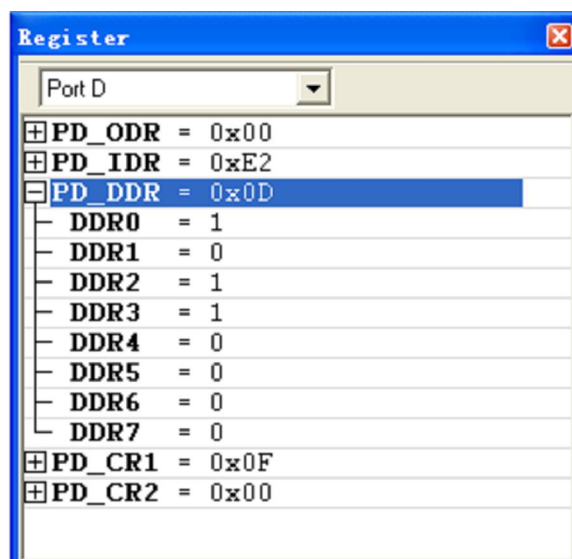
观察窗口

选择 View>Watch, 打开观察窗口，如图：



寄存器窗口

选择 View>Register，打开寄存器窗口。



5.2.4 内嵌汇编

在 C 语言中嵌入汇编为

```
asm( "nop" );
```

5.3 本章小结

本章介绍了 IAR For STM8 集成开发环境。通过具体的例子，展示了 IAR For STM8 的使用过程和主要功能。希望读者通过本章的学习，能够初步学会使用 IAR，帮助自己学习和开发 STM8 系统。

风驰电子祝您学习愉快!!! ~~~